

# LRDG: Local Replanning Dynamic Graph

Armando Palermo

**Abstract**—Il progetto prevede la simulazione, in ambiente ROS1, di un ripianificatore locale della traiettoria di inseguimento di un robot mobile, con l'obiettivo di evitare ostacoli dinamici presenti lungo un percorso globale predefinito. L'approccio si basa su una mappa nota dell'ambiente, sulla quale è stato definito un grafo utilizzato per la pianificazione e l'esecuzione dei movimenti del robot.

## I. INTRODUZIONE

Negli ultimi anni si utilizzano diversi approcci per pianificare le traiettorie che un robot deve seguire al fine di compiere un determinato compito. Gli aspetti più importanti da gestire riguardano la corretta rappresentazione della mappa, qualora disponibile, e la capacità del robot di reagire alla presenza di ostacoli dinamici.

Per questo motivo si introduce una distinzione sostanziale tra *global* e *local* planning. Nel primo caso, la pianificazione del percorso viene tipicamente effettuata offline, con l'obiettivo di determinare un percorso che consenta al robot di raggiungere il proprio goal. In particolare, vengono descritti approcci di *cell decomposition*, in cui la mappa viene suddivisa in celle, e algoritmi classici basati su euristiche, come A\*, che operano su una rappresentazione a grafo dell'ambiente [1].

La pianificazione locale, invece, si occupa della reazione del robot alla presenza di ostacoli inaspettati che possono manifestarsi all'interno di una mappa precomputata. Anche in questo caso vengono adottati approcci consolidati, tra cui il *Dynamic Window Approach (DWA)*, che considera una finestra temporale limitata all'interno della quale valuta le possibili velocità ammissibili che il robot può assumere per progredire verso il goal evitando collisioni con gli ostacoli, e il *Timed Elastic Band (TEB)*, che introduce il concetto di banda elastica temporizzata [2] [3].

L'obiettivo del progetto è quello di implementare un replanner locale che modifichi il percorso del robot in relazione ad ostacoli inaspettati, agendo esclusivamente sulla porzione di grafo che sta attraversando. L'approccio adottato si ispira alle *PRM (Probability RoadMap)* nelle quali si genera un grafo globale tramite campionamento probabilistico dei nodi [4]. Nel contesto del seguente progetto, questa metodologia viene utilizzata a livello locale, utile a definire un percorso di avoidance dell'ostacolo. Questo tipo di approccio è stata esplorata anche nel contesto della pianificazione del moto di manipolatori robotici, dove la roadmap viene generata solo nella regione di interesse al fine di garantire una risposta reattiva dal sistema [5].

## II. DESCRIZIONE DEL SISTEMA

Il sistema è implementato in diversi nodi ROS che cooperano per garantire la corretta navigazione del robot nella mappa

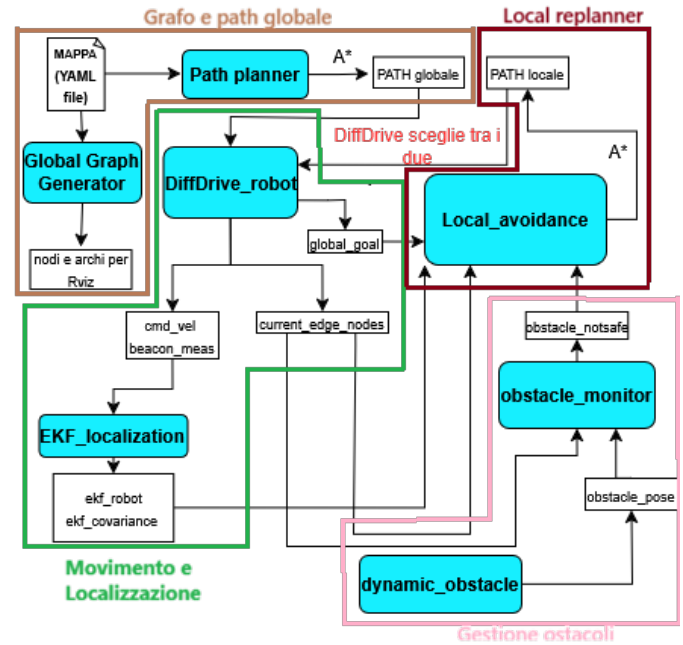


Fig. 1. Architettura del sistema e interazione tra i nodi ROS implementati.

predefinita. Alcuni di essi non sono legati alla pianificazione ma vengono utilizzati per supporto al sistema, ad esempio il nodo che genera dinamicamente gli ostacoli. L'architettura è quella mostrata nella seguente immagine.

Il flusso di funzionamento del sistema è il seguente. Tutto ha origine da un file YAML che contiene la discretizzazione della mappa sotto forma di grafo. Tramite il nodo Global Graph Generator, il contenuto del file viene convertito e pubblicato su appositi topic sotto forma di messaggi di tipo Marker, utilizzati per la visualizzazione in Rviz.

Il nodo Path planner, mediante l'algoritmo A\*, definisce il percorso che il robot deve seguire sul grafo al fine di raggiungere la destinazione desiderata.

Il nodo DiffDrive\_robot si occupa del movimento del robot lungo il path attivo, che può essere globale o locale. Inoltre, fornisce messaggi contenenti informazioni relative al controllo applicato al robot, alle misurazioni provenienti dai beacon presenti nella mappa e ai nodi iniziale e finale dell'arco che il robot sta percorrendo.

La stima della posa del robot viene effettuata dal nodo `EKF_localization`, che sfrutta le misurazioni dei beacon e il modello di moto del robot.

Per simulare la presenza di ostacoli inaspettati che intersecano il percorso globale definito dal planner, viene utilizzato il nodo `dynamic_obstacle`.

Il nodo `obstacle_monitor` fornisce informazioni riguardo la pericolosità dell'ostacolo individuato rispetto all'arco che il robot sta attraversando.

`Local_avoidance` rappresenta il nodo centrale dell'architettura e si occupa della generazione di un grafo locale e del corrispondente path alternativo, in grado di evitare l'ostacolo inaspettato.

Nella sezione successiva vengono analizzati nel dettaglio i singoli nodi appena descritti.

### III. IMPLEMENTAZIONE DEI NODI

In questa sezione vengono analizzati i nodi dell'architettura presentata, con maggior focus sui nodi principali.

#### A. Global Graph Generator

Il Global Graph Generator è il nodo che si occupa della trasposizione dal file YAML alla visualizzazione in RViz della mappa discretizzata sotto forma di grafo.

```
1 map:
2   frame_id: "map"
3   size: [50.0, 50.0]
4
5 nodes:
6   - id: 0
7     x: -18.0
8     y: -15.0
9
10  - id: 1
11    x: -10.5
12    y: -17.2
13
14  - id: 2
15    x: -2.0
16    y: -14.0
17
```

Fig. 2. Prime righe della mappa in formato YAML

Il grafo di navigazione è definito nel sistema globale `map`, infatti le coordinate dei nodi sono espresse in tale frame. Le dimensioni della mappa vengono utilizzate esclusivamente per delimitare l'area di pianificazione.

Il nodo in questione genera l'insieme dei nodi del grafo a partire dal file YAML e, successivamente, costruisce gli archi del grafo in base a un iperparametro che ne definisce la distanza massima consentita (`max_distance_edges`).

Sia i nodi che gli archi vengono pubblicati su appositi topic ROS mediante messaggi di tipo `Marker`, consentendone la visualizzazione in RViz.

Il grafo risultante è un grafo statico.

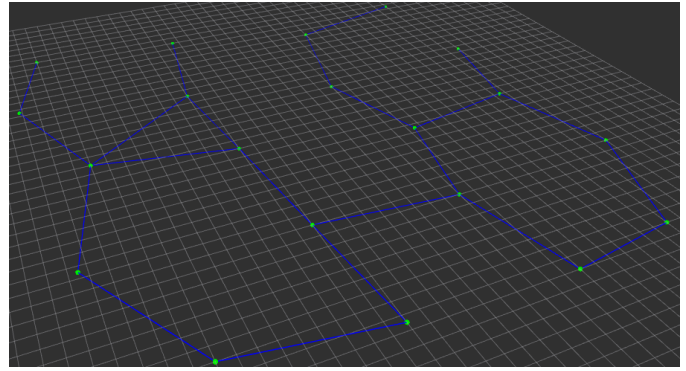


Fig. 3. Grafo globale visualizzato in RViz

#### B. Path Planner

Il nodo Path Planner è quello che si occupa della definizione del percorso globale che il robot deve seguire. Al suo interno vengono definiti due nodi del grafo, `start_node`, ovvero il punto di partenza del robot, ed `end_node`, ovvero il punto di arrivo.

Il grafo globale viene costruito a partire da un insieme di nodi letti da un file YAML. Tali nodi sono rappresentati come coordinate bidimensionali e usati per costruire una struttura a grafo, in maniera analoga a quanto fatto dal Global Graph Generator. In questo caso la struttura risultante è un dizionario che associa ad ogni nodo l'elenco dei nodi vicini e i relativi costi di percorrenza degli archi che li collegano.

Tale grafo viene fornito in ingresso alla funzione `astar`, che utilizza come euristica la distanza euclidea tra il nodo corrente e il nodo di arrivo, al fine di stimare il costo rimanente e determinare un percorso globale ottimo che soddisfi il task richiesto.

Nella figura successiva è mostrato il risultato del path planner. In alto a sinistra si trova il nodo iniziale.

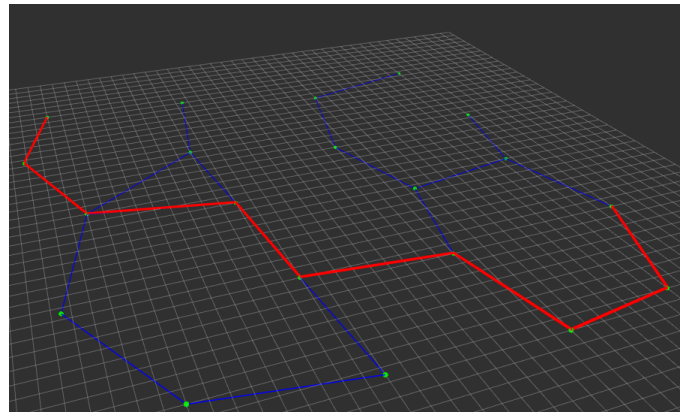


Fig. 4. Path globale visualizzato in RViz

### C. EKF localization

Per la stima della posa del robot all'interno della mappa viene utilizzato un filtro di Kalman esteso (EKF), che fonde il modello di moto Differential Drive del robot con misurazioni di distanza provenienti da beacon a posizione nota.

Lo stato del sistema è definito dalla posizione e dall'orientamento del robot:

$$\mathbf{x} = [x \quad y \quad \theta]^T$$

a) *Fase di predizione:* Nella fase di predizione, la posa stimata viene aggiornata tramite il modello di moto del robot, mentre la covarianza viene propagata utilizzando i Jacobiani del modello rispetto allo stato e agli ingressi di controllo:

$$\begin{aligned} \mathbf{x}_k^- &= f(\mathbf{x}_k, \mathbf{u}_k) \\ P_k^- &= F_x P_k F_x^T + F_u Q F_u^T \end{aligned} \quad (1)$$

dove il Jacobiano del modello di moto rispetto allo stato è definito come:

$$F_x = \frac{\partial f(\mathbf{x}, \mathbf{u})}{\partial \mathbf{x}} = \begin{bmatrix} 1 & 0 & -v \sin \theta \Delta t \\ 0 & 1 & v \cos \theta \Delta t \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

mentre il Jacobiano rispetto ai comandi di velocità lineare e angolare è:

$$F_u = \frac{\partial f(\mathbf{x}, \mathbf{u})}{\partial \mathbf{u}} = \begin{bmatrix} \cos \theta \Delta t & 0 \\ \sin \theta \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \quad (3)$$

La matrice di rumore di controllo è modellata come:

$$Q = \begin{bmatrix} \alpha_v v_k^2 & 0 \\ 0 & \alpha_\theta \omega_k^2 \end{bmatrix} \quad (4)$$

b) *Fase di correzione:* Nella fase di correzione, il filtro confronta le misurazioni di distanza dai beacon con le distanze attese, calcolate a partire dalla posa predetta. Le posizioni dei beacon sono assunte note e fanno parte della mappa, ma non del grafo di navigazione. L'innovazione risultante viene utilizzata per aggiornare la stima dello stato e ridurre l'incertezza associata.

Il modello di misura associato al beacon  $i$ -esimo è definito come:

$$h_i(\mathbf{x}) = \sqrt{(x - x_{b_i})^2 + (y - y_{b_i})^2} \quad (5)$$

Il Jacobiano del modello di misura è quindi:

$$H_i = \frac{\partial h_i(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{x - x_{b_i}}{d_i} & \frac{y - y_{b_i}}{d_i} & 0 \end{bmatrix}, \quad (6)$$

$$d_i = \sqrt{(x - x_{b_i})^2 + (y - y_{b_i})^2}$$

La fase di aggiornamento del filtro è quindi descritta dalle seguenti equazioni:

$$\begin{aligned} \mathbf{y}_k &= \mathbf{z}_k - \hat{\mathbf{z}}_k \\ S_k &= H P_k^- H^T + R \\ \mathbf{x}_k &= \mathbf{x}_k^- + K_k \mathbf{y}_k \\ P_k &= (I - K_k H) P_k^- \end{aligned} \quad (7)$$

dove la matrice di rumore di misura è definita come:

$$R = \sigma_z^2 I \quad (8)$$

La posa stimata del robot e la relativa covarianza vengono infine visualizzate in RViz. Queste vengono pubblicate sui seguenti topic ROS: /ekf\_robot e /ekf\_covariance.

### D. dynamic\_obstacle

Nodo che si occupa della generazione di ostacoli improvvisi lungo il percorso che il robot deve inseguire. Lo scopo di tali ostacoli è quello di verificare la reattività del local replanner.

Il nodo li pubblica in maniera ciclica, ogni quattro secondi, sul topic /dynamic\_obstacle, tramite messaggi marker contenenti la posa e il raggio dell'ostacolo. Per semplicità gli ostacoli sono rappresentati come dei cilindri

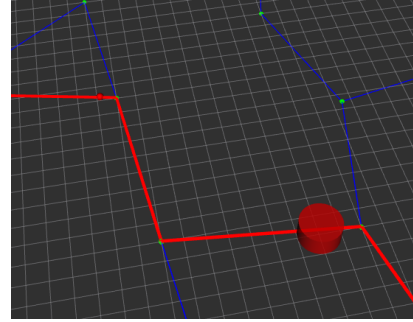


Fig. 5. Esempio ostacolo visualizzato in RViz

### E. obstacle\_monitor

Il nodo obstacle\_monitor si occupa di segnalare la presenza di un ostacolo solo nel caso in cui esso risulti pericoloso per il robot mentre sta attraversando un determinato arco del percorso.

Dai topic dynamic\_obstacle e current\_edge\_nodes (quest'ultimo pubblicato da un nodo che verrà analizzato successivamente) vengono acquisite tutte le informazioni necessarie per valutare la pericolosità dell'ostacolo. Dal primo viene estratta la posizione dell'ostacolo nel frame map, mentre dal secondo viene ricavato l'arco corrente lungo il quale il robot si sta muovendo.

Viene quindi calcolata la distanza punto-segmento tra l'ostacolo e l'arco considerato. Nel caso in cui l'ostacolo intersechi l'arco, esso viene classificato come pericoloso e la sua posa viene pubblicata sul topic obstacle\_not\_safe.

## F. Local avoidance

Questo è il nodo cardine dell'intero progetto e si occupa della generazione di un grafo locale utile all'aggiramento dell'ostacolo inaspettato.

Il nodo si sottoscrive ai seguenti topic: `obstacle_notsafe`, `ekf_robot`, `current_edge_nodes` e `/next_global_node`.

In un contesto nel quale si presenta un ostacolo improvviso sull'arco corrente che il robot sta attraversando viene richiamato il metodo `avoid_collision`. Questo permette di definire un percorso alternativo che il robot deve inseguire.

Come prima cosa viene definita la finestra di azione del replanner, il cui centro è identificato dal punto medio dell'arco corrente e il cui lato dipende dalla posizione e dalla dimensione dell'ostacolo.

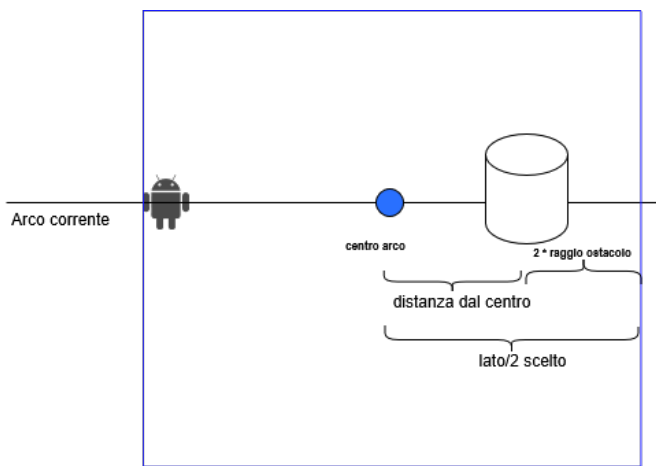


Fig. 6. finestra di azione del local avoidance node

Dopo aver definito la finestra di azione del nodo vengono generati randomicamente dei nodi in questa finestra tramite una distribuzione uniforme. Il numero di nodi generati è definito dalla variabile `n_of_random_nodes`. Non tutti i nodi così generati vengono presi in considerazione, infatti, si filtrano quelli che ricadono sull'ostacolo.

Una volta definito l'insieme dei nodi, vengono generati gli archi tra essi, tenendo conto della distanza massima possibile, ed evitando di generarli quando questi intersecano l'ostacolo.

Vengono definiti due nodi principali che sanciscono i limiti del percorso locale definito dal grafo appena creato. Il nodo `start` viene generato in corrispondenza della posizione attuale del robot, mentre il nodo `goal` è il nodo finale dell'arco globale su cui è stato individuato l'ostacolo. Questi ultimi sono collegati ai nodi più vicini tra quelli generati nella finestra randomica.

Tramite il grafo risultante e la funzione `astar` viene quindi definito il percorso ottimo sul grafo locale e pubblicato su topic utili alla visualizzazione e all'inseguimento del robot.

## G. DiffDrive\_robot

È il nodo responsabile del movimento del robot. Resta in ascolto sui topic `/global_graph_path/edges` e `/local_path`, che contengono messaggi di tipo `Marker`

e rappresentano, rispettivamente, il percorso globale e quello locale.

Inoltre, il nodo pubblica messaggi relativi alle misurazioni dei beacon, ai comandi di velocità del robot, alle informazioni sull'arco corrente attraversato (sia globale che locale) e al termine di una fase di replanning locale o durante l'esecuzione del percorso globale, pubblica le informazioni relative al prossimo nodo globale da raggiungere.

Di default il robot si muove in relazione al path globale, selezionando come nodo da raggiungere il secondo nodo dell'arco su cui si trova. È stata usata una legge di controllo semplice, nella quale la velocità lineare è costante, mentre quella angolare è proporzionale alla differenza tra l'orientamento del robot e la direzione verso il target.

Quando si presenta un ostacolo inaspettato sul percorso, viene generato il path locale dal local replanner, e in corrispondenza della lettura di tale messaggio, viene settato a true un flag (`using_local`) che indica al nodo di non considerare il path globale ma quello locale.

Il metodo principale è `step`, richiamato periodicamente, nel quale come prima cosa viene scelto il path su cui il robot deve avanzare sulla base dell'attributo di cui sopra. Dopodiché a seconda del nodo di arrivo vengono definite le velocità atte a raggiungerlo. Infine, pubblica sui topic relativi alle misurazioni dei beacon e dei comandi di velocità.

## IV. ANALISI DEI RISULTATI

Dopo aver compilato il pacchetto, tramite il seguente comando vengono lanciati tutti i nodi di cui sopra.

```
roslaunch graph_navigation launch.launch
```

La posizione iniziale del robot coincide con il nodo `start` definito per il percorso. Quando viene rilevato un ostacolo lungo l'arco che il robot sta attraversando, viene generato un grafo locale su cui viene individuato un percorso in grado di evitare l'ostacolo. Tale percorso locale si ricongiunge poi con quello globale in corrispondenza del secondo nodo dell'arco originario.

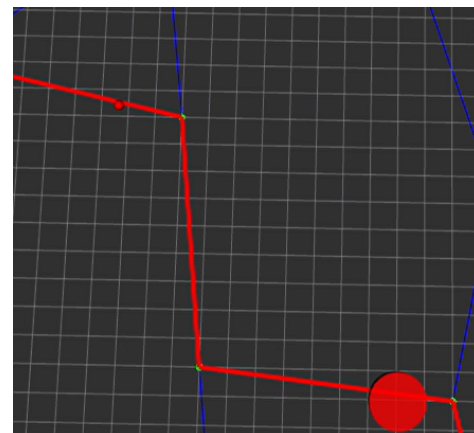


Fig. 7. Il robot (pallino rosso in alto a sx) avanza verso gli archi su cui compaiono gli ostacoli



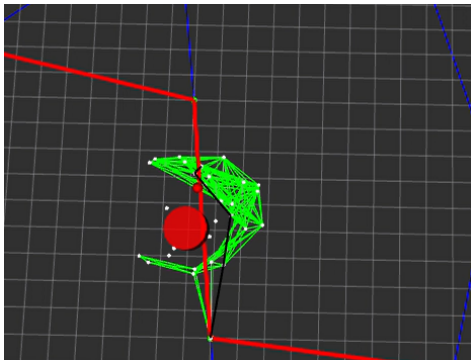


Fig. 8. Generazione percorso locale

Attualmente il replanner gestisce anche un secondo ostacolo che si palesa su un arco locale. L'approccio in questo caso è il medesimo di cui sopra. La sola variazione consiste nell'analizzare come *current.edge* l'arco locale invece di quello globale. Anche in questo caso il nodo di ricongiunzione del grafo locale è il secondo nodo dell'arco globale.

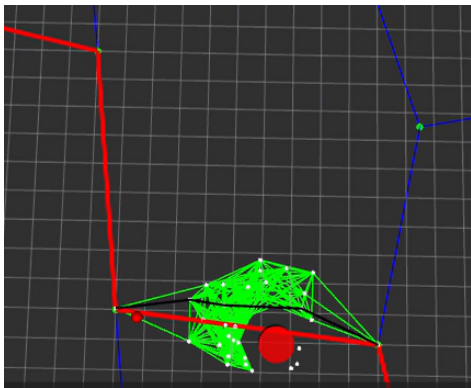


Fig. 9. Generazione percorso locale primo ostacolo

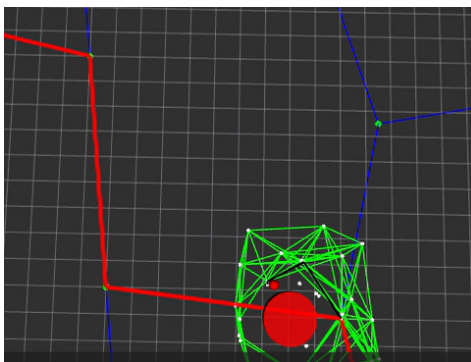


Fig. 10. Generazione percorso locale secondo ostacolo

## V. CONCLUSIONI E SVILUPPI FUTURI

Il replanner proposto ha dimostrato di gestire correttamente la presenza di ostacoli improvvisi lungo il percorso del robot; tuttavia, sono presenti alcune limitazioni che costituiscono possibili spunti per sviluppi futuri.

I test sono stati condotti in un ambiente simulato che considera ostacoli puntuali e di forma semplice, senza modellarne la dinamica. Un possibile sviluppo consiste nell'introdurre

meccanismi di stima o previsione del moto degli ostacoli, al fine di ottimizzare il percorso locale di avoidance.

Il grafo locale è generato in modo non deterministico, a causa della natura stocastica del campionamento dei nodi nella finestra di pianificazione, producendo talvolta soluzioni differenti a parità di configurazione dell'ambiente.

Il numero di nodi influisce direttamente sulla complessità computazionale dell'algoritmo. Per limitarne l'impatto, i nodi di start e goal vengono connessi unicamente al nodo più vicino; in alcune situazioni, ciò può impedire il ricongiungimento con il grafo globale. Strategie di connessione più robuste o un aumento controllato dei nodi locali rappresentano possibili miglioramenti.

L'approccio assume inoltre la disponibilità di una percezione globale dell'ambiente, necessaria per valutare la distanza tra gli ostacoli e gli archi del percorso globale. In uno scenario reale, tale informazione non è direttamente accessibile a un robot dotato di soli sensori locali; l'impiego di sensori globali distribuiti, come telecamere fisse o sistemi di tracking esterni, rappresentano possibili estensioni.

Infine, la legge di controllo adottata, intenzionalmente semplice, può evidenziare alcuni limiti dell'approccio di pianificazione locale. In particolare, il replanner può generare traiettorie non continue, con un primo arco orientato in modo significativo rispetto alla direzione del robot, producendo comportamenti transitori poco efficienti. L'inclusione dell'orientamento del robot nella generazione del grafo locale potrebbe mitigare tali effetti.

## USE OF GENERATIVE AI TOOLS

Strumenti di intelligenza artificiale generativa (ChatGPT) sono stati utilizzati come supporto alla stesura di alcune porzioni di codice. In particolare, tali strumenti sono stati impiegati per la generazione dei messaggi *Marker* per la visualizzazione in *RViz*, nella stesura dell'algoritmo *A\**, e per la scrittura del nodo dedicato alla generazione casuale di ostacoli nell'ambiente simulato.

## REFERENCES

- [1] M. Badamasi Aremu, I. K. Kabir, G. Ahmed, and S. El-Ferik, "Autonomous mobile robot path planning techniques—a review: Classical and heuristic techniques," *IEEE Access*, vol. 13, pp. 117 999–118 022, 2025.
- [2] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [3] C. Rösmann, F. Hoffmann, and T. Bertram, "Integrated online trajectory planning and optimization in distinctive topologies," *Robotics and Autonomous Systems*, vol. 88, pp. 142–153, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889016300495>
- [4] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [5] Y. Liu, B. Chen, X. Zhang, and R. Li, "Research on the dynamic path planning of manipulators based on a grid-local probability road map method," *IEEE Access*, vol. 9, pp. 101 186–101 196, 2021.